



链接库

▼ 编译器

▼ GCC (GNU Compiler Collection)

由 GNU 开发，支持多种操作系统，包括 Linux、Windows（通过 MinGW 或 Cygwin）、macOS 等。

GCC 是一个多语言编译器，支持 C、C++、Fortran 等多种语言。

通常与 GDB（GNU Debugger）配合使用。工具链还包括其他工具，如 Make、Autotools 等。

▼ MinGW (Minimalist GNU for Windows)

为 Windows 系统设计的开发环境，提供了一组 GCC 工具链的 Windows 端口。它允许开发者在 Windows 上使用 GCC 来编译 Windows 应用程序。

▼ G++ (GCC for C++)

G++ 专门用于编译 C++ 代码，本质上是 GCC 的一个变种，用于更好地支持 C++ 的特性和标准库。

默认情况下，GCC 假设输入文件是 C 语言（如果文件后缀名为 `.c`），并使用 C 语言编译器。如果需要编译 C++ 代码，必须显式指定 `g++` 或者使用 `-x c++` 选项。

在编译 C++ 代码时，GCC 不会自动链接 C++ 标准库（libstdc++）。因此，如果使用 GCC 编译 C++ 代码，通常需要手动指定链接选项（如 `-lstdc++`）。

而 G++ 在默认情况下，假设输入文件是 C++ 语言（如果文件后缀名为 `.cpp` 或 `.cc` 等），并使用 C++ 语言编译器。

并且 G++ 会自动链接 C++ 标准库。因此，使用 G++ 编译 C++ 代码时，不需要手动指定链接 C++ 标准库。

▼ MSVC (Microsoft Visual C++)

MSVC 是 Microsoft Visual Studio 中的 C++ 编译器，是 Windows 开发的主流工具。

MSVC 提供类似的优化选项（如 /O1、/O2、/Ox），但选项和命令行语法与 GCC 不同。MSVC 也提供了一些 Windows 平台特有的优化。

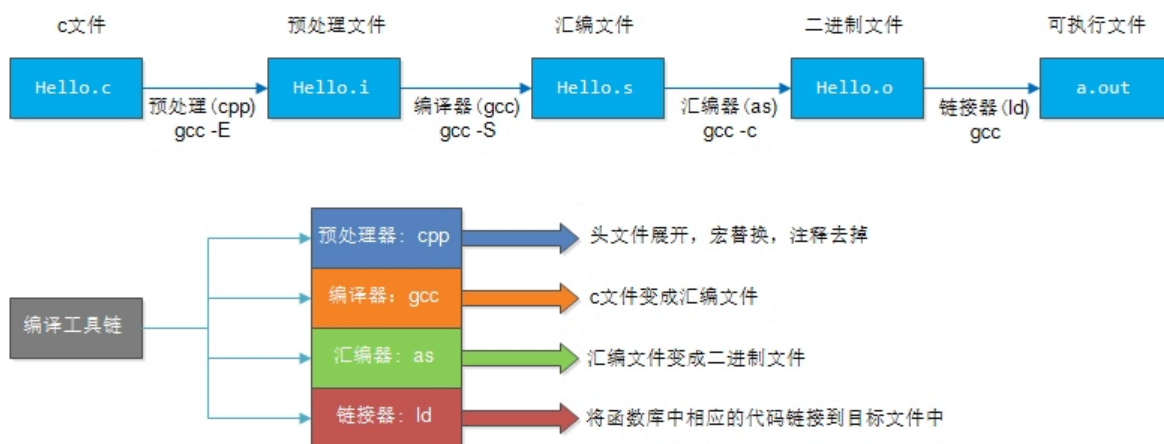
MSVC 集成在 Visual Studio IDE 中，支持各种插件和扩展，提供丰富的调试功能和集成工具，如 Profiler、Memory Checker 等。

▼ 编译过程

▼ 总体流程

一个 C/C++ 文件要经过预处理(preprocessing)、编译(compilation)、汇编(assembly)和链接(linking)4 步才能变成可执行文件。

程序代码首先经过预处理器生成 .i 文件。然后通过编译器生成 .s 汇编文件，再通过汇编器生成 .o 目标文件，最后通过链接器链接生成可执行文件。



其中的编译阶段消耗的时间和系统资源最多。

▼ 预处理

预处理阶段是编译前的准备工作主要进行一下几个工作：

- 执行预处理指令
- 将头文件展开 即 `#include` 所包含的内容
- 进行宏替换
- 删除注释
- 添加行号和标识

预处理器会把 `#include` 指令指定的头文件内容直接插入到包含它的源文件（`.cpp`）中，形成一个整体的编译单元。例如，`#include <iostream>` 会将 `iostream` 头文件的所有内容插入到当前文件。这种文件包含是文本替换，直接扩展源代码。

所以源文件与其包含的头文件会被算作同一个编译单元。可以认为一个编译单元包含一个源文件与其包含的头文件。

为了防止头文件被重复包含（例如，当多个文件都 `#include` 相同的头文件时），通常在头文件中会使用头文件保护（Include Guard）或 `#pragma once`。

该保护只是在同一个编译单元中起作用，不同的源文件会分别处理自己的包含路径。所以第二个被编译的源文件中会再次展开该头文件。

预处理阶段是各个源文件独自进行的，每个源文件互不干扰，预处理的作用范围仅在本文件中，在预处理完成后，会生成一个.i 文件，而这个.i 文件就是下一阶段编译时的一个编译单元，在此之后头文件就已经没有任何的作用了

▼ 编译

编译是以编译单元为单位基本单位进行的，而编译单元即为预处理结束后每个.cpp 文件生成的.i 文件。编译时有以下工作

- 生成符号表
- 函数和变量声明的检查
- 语法分析 语义分析 词法分析 符号汇总等
- 将代码转换为汇编语言

函数声明可以告诉编译器某个函数的存在以及它的返回类型和参数类型，编译器据此知道如何调用该函数，即使函数的实现在其他文件中。

类的声明或定义允许编译器理解类的结构和成员的类型。头文件中的类通常会包括成员函数的声明（但不包括具体实现）。

如果成员函数的实现不在头文件中，而是在源文件中，则编译器仅在编译时需要类的布局信息，而不需要具体实现内容。

如果头文件中包含模板类或函数，则它们的实现通常也包含在头文件中，因为模板需要在编译时生成代码。

不过编译的主要工作还是以下部分：

1. 词法分析（Lexical Analysis）

- 将代码拆分为基本的语法单元（如关键字、变量名、运算符、标点符号等）。

2. 语法分析（Syntax Analysis）

- 生成语法树，检查语法是否正确，比如是否存在遗漏的分号、未匹配的括号等。

3. 语义分析（Semantic Analysis）

- 在这一阶段，编译器会执行更深层次的检查，例如：
 - 变量类型是否匹配。
 - 类、函数、变量的作用域是否正确。
 - 函数的使用方法是否与声明（以及同一编译单元内的定义）的类型一致。

这一过程并不涉及到其他编译单元的函数定义，只会根据声明检测使用方法是否正确，除非该函数的定义与声明在同一编译单元。

- 类型转换和隐式转换是否合法。
- 抽象类是否被不合法地实例化。

4. 代码生成 (Code Generation)

- 将符合语法和语义的代码生成目标机器代码或中间代码（如汇编代码）。

编译时仅对函数和变量的声明进行检查，而不关心函数的定义，编译是以一个个单独的文件为单元的，与预处理一样，而这些编译单元都是之前生成的.i 文件。编译完成后会生成.s 文件

▼ 汇编

将编译后生成的.s 文件进行汇编

- 转成机器指令
- 生成包含机器代码、数据段和符号表等信息的目标文件

在汇编阶段，符号（如变量名和函数名）仅被记录为**逻辑地址/虚拟地址**，即它们在目标文件中的引用并没有被具体的物理地址替换。

汇编过程不会处理符号的最终地址，只是将符号记录在符号表中。

在经过汇编阶段后，每个.s 会生成一个对应的二进制的.o 文件

▼ 链接

这一部分会将所有编译单元整合起来，所以**所有跨编译单元的操作都在这一部分发生**，比如不同编译单元中同一函数的声明与定义的匹配。

- 符号地址重定位
- 将.o 目标文件链接起来生成可执行文的.exe 件。

链接器会负责匹配函数的声明和定义，**尤其处理跨文件的符号解析**。

在链接过程中，首先会对所有目标文件中的符号进行解析。符号可以是函数、变量、类等名称，链接器会扫描每个目标文件，记录下所有的符号及其引用关系：

- **定义符号**：在某个目标文件中定义的符号，例如一个函数的实现。
- **引用符号**：在某个目标文件中使用但未定义的符号，例如调用另一个文件中的函数。

链接器需要识别哪些符号是定义的，哪些符号是需要解析的。

一旦符号解析完成，链接器会开始进行地址重定位。这是将符号的逻辑地址转换为最终的物理地址的过程：

- **符号地址分配**：链接器为每个定义的符号分配一个地址。这个地址通常是目标文件中的段地址（如代码段、数据段等）。
- **重定位条目**：目标文件中包含重定位信息，这些信息告诉链接器如何修正符号引用的地址。

在此过程中，链接器会根据符号在目标文件中的位置，更新所有引用该符号的地方，使其指向正确的地址。

在链接阶段之前，各个编译单元都是相互独立的，互不干扰。

地址的重定位就是建立物理内存与虚拟地址间的映射。

在链接阶段，链接器会对所有目标文件中的符号进行解析和处理。链接器为每个定义的符号分配一个实际的物理地址，这个地址是基于目标文件中代码和数据的排列而决定的。

当链接器解析完所有符号后，它会修正目标文件中的符号引用，将它们的逻辑地址替换为链接器为它们分配的物理地址。

如果头文件中存在定义，并被多个源文件包含，则会在该过程出现重定义的错误。

▼ 符号表

在 C++ 编译器的编译过程中，符号表生成在编译阶段，有以下两个主要过程：

1. 词法分析和语法分析阶段（前端分析）

在编译器的前端，词法分析器和语法分析器会解析源代码并识别出各个符号（如变量名、函数名、类名等）。

- 词法分析将代码拆解为单词（Tokens）。
- 语法分析根据语法规则构建抽象语法树（AST，Abstract Syntax Tree）。
- 在此过程中，符号表会逐步生成，用于记录符号的名称、作用域、类型等信息。

2. 语义分析阶段

在语义分析阶段，编译器检查符号的上下文是否合理（例如，变量是否声明、函数调用的参数是否匹配等）。

- 符号表会在这个阶段得到进一步完善，包括添加符号的类型、修饰符、作用域、存储位置等信息。
- 编译器也会通过符号表检测重定义、未定义符号等问题。

符号表的内容

符号表通常包含以下信息：

- 符号的名称（变量、函数、类等）。
- 符号的类型（基本类型、指针类型、类类型等）。
- 符号的作用域（全局作用域、局部作用域、类作用域等）。
- 符号的存储位置（栈、堆或全局区域）。
- 符号的修饰符（如 `static`、`const`、`virtual` 等）。

符号表的用途

1. 检查符号是否被重复定义或使用未定义的符号。
2. 支持作用域解析和类型检查。
3. 链接阶段借由符号表，将存在调用关系的不同文件链接，用于生成目标代码。

▼ 外部库链接

在编译阶段，程序会为外部库中的函数预留空间，直到链接阶段再将外部库的函数填入。

库文件本质是由 cpp 文件生成，其需要搭配对应的头文件使用。可以将库文件理解为实现了头文件中函数与类定义的 cpp 文件的封装。

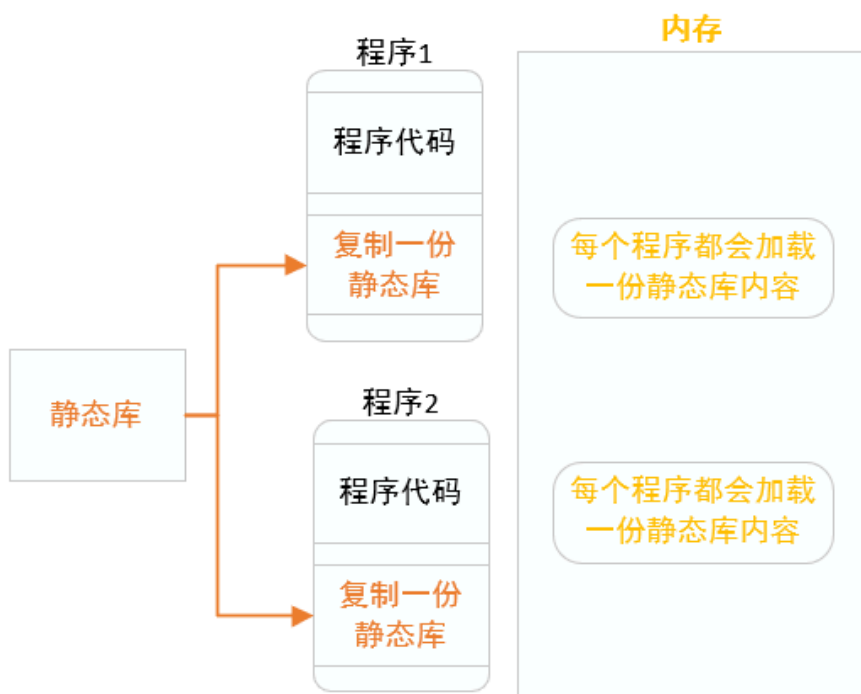
在链接阶段，根据链接过程中处理方式的不同，链接可以分为静态链接和动态链接。

▼ 静态链接

静态链接是在链接阶段，就把所有需要的函数的二进制代码都包含到可执行文件中去，完成所有符号引用的一种链接方式。

所以静态链接在编译完成后，就不再需要保留静态库的文件，而动态链接不仅需要保存文件，还需要将其路径加入环境变量，确保操作系统能够找到需要的依赖库。

作为代价，就是程序的内存消耗增大。每一份使用到静态库的文件，都会完整的复制一份静态库文件。即使使用的只是静态库中的一个函数，也会完整复制整份静态库。



在使用静态链接时，为了减少内存消耗，需要尽可能减少使用静态库的文件数量，并且将静态库尽可能拆分成多份，避免因为一个函数导入太多不需要的内容。

静态库被打包到可执行程序中，生成的可执行文件较大，但是程序加载运行速度会比较快，发布程序时，也无需提供静态库，移植比较方便。

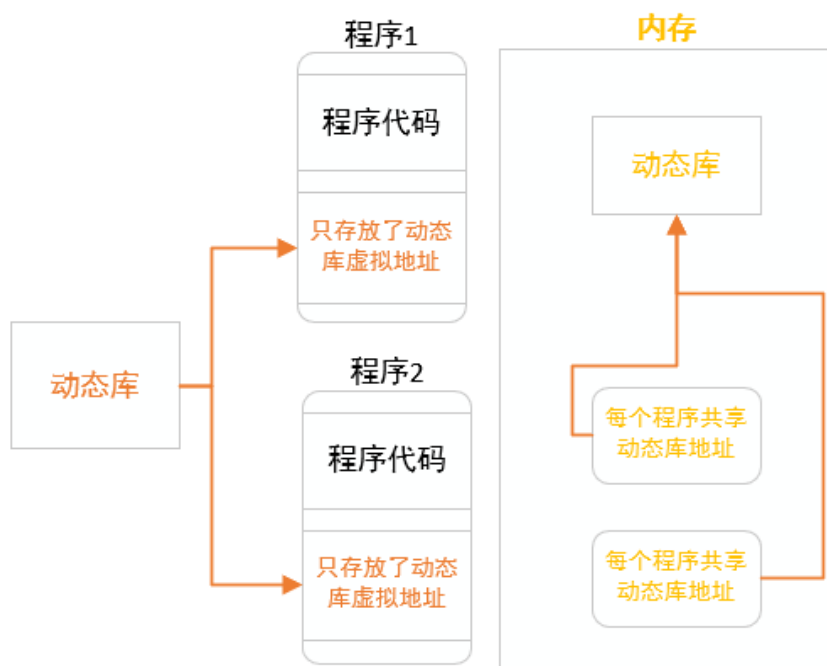
但是当多个程序链接同一个静态库时，生成的每一个可执行文件中，都会含有这个静态库，相当于在内存中同时运行着两个相同的静态库，比较浪费系统资源。

另外，当静态库的内容发生更新时，依赖该静态库的程序也需要重新进行链接，导致程序的更新升级会比较麻烦。

▼ 动态链接

动态链接在链接的时候不直接拷贝相应依赖库的代码，而是通过记录一系列符号和参数，在程序运行或加载时将这些信息传递给操作系统，由操作系统负责将需要的依赖库加载到内存中，然后程序在运行到指定代码时，去执行内存中已经加载的依赖库的代码，最终达到运行时链接的目的。

动态库则不会将代码打包到可执行文件中，而是打包动态库的名称等信息，在可执行程序运行的时候，需要去找到动态库的文件，然后把动态库加载到内存中才可以使用动态库中的代码。



动态库可以实现进程间资源共享，比如可执行程序 A 在运行时用到了某个动态库，那么系统会将该动态库的代码动态加载到内存中，此时若正在运行的程序 B 也需要用到该动态库中的代码，则不需要再次加载该动态库，而是可以与程序 A 共享。

此外，当动态库的内容更新时，只需重新编译生成新的动态库即可，而不需要对依赖该动态库的程序重新进行编译链接。

▼ 动态库与静态库的区别



无论是动态库 (`.dll` / `.so`) 还是静态库 (`.lib` / `.a`), 都会有 `.lib` 文件

类型	<code>.lib</code> (Windows 上) 作用	其他文件	链接时发生什么	程序运行时需要什么
静态库	真正的库内容, 编译好的代码	没有其他文件	链接时, 直接把 <code>.lib</code> 中的二进制代码拷贝进你的程序	程序无需额外文件
动态库	仅是导入库 (Import Library), 里面是函数符号信息 (地址在 DLL 里), 只存放 <code>.dll</code> 中符号的引用信息	有 <code>.dll</code> 文件 (动态链接库)	链接时, 只记录: 将来从哪个 DLL 中找这些函数	程序运行时, 必须有对应 DLL 文件

静态库只是符号表与一堆 `.obj` 的归档, 仍属于链接前的产物, 链接时用到哪个符号, 就把对应 `.obj` 直接拷贝进最终可执行文件, 所以只要有头文件就可以使用静态库中的非 `static` 函数与类。

但是动态库不同, 其与 `exe` 文件类似, 也是属于链接完成的产物, 其内存布局也与 `exe` 文件类似, 包含栈、堆、数据段、代码段, 除此之外还有 `exe` 文件没有的导出表与 TLS。

在动态库项目中, 需要使用 `__declspec(dllexport)` 导出符号, 外部使用时则使用 `__declspec(dllimport)` 导入符号。

`__declspec(dllexport)` 导出的函数和类的符号会被记录在导入库 `lib` 中, 所以在使用 `dll` 文件时需要搭配对应的 `lib` 文件。

不使用 `__declspec(dllimport)` 导入的变量无法链接成功, 但是导入的函数有时可以链接成功, 是因为链接器会在尝试链接本地函数失败后调整链接方式, 代价是造成额外的消耗。在汇编层面,

调用本地函数的汇编指令为 `call foo`，而调用导入函数的汇编指令为 `call [__imp_foo]`，`__imp_foo` 是一个指针变量，由系统加载 DLL 时填充。

```
C++

#ifdef MYLIB_SHARED
    #ifdef MYLIB_EXPORTS
        #define MYLIB_API __declspec(dllexport)
    #else
        #define MYLIB_API __declspec(dllimport)
    #endif
#else
    #define MYLIB_API // 静态库
#endif

MYLIB_API void foo(); // 声明处加 dllexport

class MYLIB_API Foo {
public:
    void bar(); // 成员函数声明，自动导出
};
```

`__declspec(dllexport)` 和 `__declspec(dllimport)` 只需要在声明处使用即可，定义处不需要再次使用。

需要注意的是，如果是需要跨语言使用的 `dll` 文件，则最好使用 C ABI，`extern "C"`
`MYLIB_API void Foo_doSomething(FooHandle handle);`，并避免使用 C++ 对象。

▼ 库文件的编译

在 windows 中，静态库后缀为 `.lib`，动态库后缀为 `.dll`。

在 linux 中，静态库后缀为 `.a`，动态库后缀为 `.so`。

Debug 环境下编译得到的库文件会比 Release 环境下得到的库文件多一个 `d` 字母，比如一个名为 `a.lib` 的库文件，在 Debug 环境下为 `ad.lib`。

编译库文件可以使用 CMake，具体使用方法见：[CMake](#)。

使用外部库时项目环境的配置，见：[VS 项目环境配置](#)。